

## Pandas DataFrames – Creation, Indexing & Selection



Suman

18 Feb, 2026 At 8:00 PM

Notes

Module-1

Recommended

Resource



Associated Lectures (1)



Description



Discussions

### Session 4.1: Lecture Notes

[Colab File](#)[PPT File](#)

### Pandas DataFrames: Creation, Indexing & Selection

**Program:** AIM101 -- Artificial Intelligence & Machine Learning Foundation **Institution:** Vishlesan i-Hub IIT Patna x Masai School **Session Type:** Instructor (Practical, Hands-on) **Prerequisites:** Week 3 complete (NumPy: arrays, operations, indexing, boolean masking, reshaping)

### Mental Map

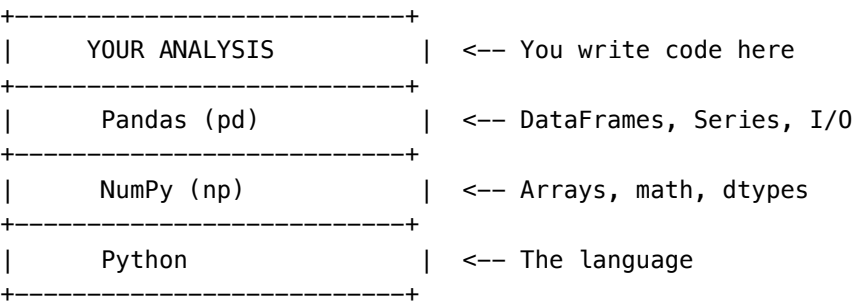
Session 4.1: Pandas DataFrames

```
|
+-- A. WHY PANDAS
|   +-- Excel Horror Stories (gene corruption, $6.2B copy-paste)
|   +-- NumPy vs Pandas comparison
|   +-- import pandas as pd
|
+-- B. SERIES (1D Labeled Array)
|   +-- Create from list, list+index, dictionary
|   +-- Attributes: .values, .index, .dtype, .shape, .name
|   +-- Access by label: s['Alice']
|   +-- Operations & boolean masking (same as NumPy, but labeled)
|
+-- C. DATAFRAME CREATION (2D Labeled Table)
|   +-- Method 1: From dictionary (keys -> columns)
|   +-- Method 2: From list of lists (rows -> need columns= param)
|   +-- Method 3: From NumPy array (add labels to numbers)
|   +-- Method 4: From list of dicts (API/JSON style)
|   +-- Attributes: .shape, .columns, .index, .dtypes, .values, .size, .ndim
```

```
|
+-- D. READING & EXPLORING DATA
|   +-- pd.read_csv() with parameters (sep, usecols, nrows, dtype)
|   +-- Five-Method Exploration Ritual:
|       .head(), .tail(), .shape, .info(), .describe()
|
+-- E. COLUMN SELECTION
|   +-- Single: df['col'] -> Series
|   +-- Multiple: df[['col1','col2']] -> DataFrame
|   +-- Dot notation pitfalls (spaces, method names, variables)
|   +-- df['col'] vs df[['col']] (Series vs DataFrame)
|
+-- F. ROW SELECTION (CRITICAL)
|   +-- .loc[] = Label-based, INCLUSIVE slicing
|   +-- .iloc[] = Integer position, EXCLUSIVE slicing
|   +-- Combined: .loc[row, col] / .iloc[row, col]
|   +-- Chained indexing warning (never df['col'][row])
|
+-- G. BOOLEAN FILTERING
|   +-- Single: df[df['col'] > val]
|   +-- Multiple: & (and), | (or), ~ (not) -- PARENTHESES mandatory
|   +-- .query("condition") -- cleaner syntax
|
+-- H. SORTING
|   +-- .sort_values(by='col', ascending=False)
|   +-- .sort_index()
|   +-- .reset_index(drop=True)
|   +-- Clean pipeline: filter -> sort -> reset
```

# The Big Picture

## The Python Data Stack



## When to Use Which

| Use NumPy When...                 | Use Pandas When...            |
|-----------------------------------|-------------------------------|
| Pure numerical computation        | Tabular data with mixed types |
| Matrix operations, linear algebra | Need labeled columns/rows     |
| Performance-critical math         | Reading CSV/Excel files       |

| Use NumPy When...                | Use Pandas When...               |
|----------------------------------|----------------------------------|
| Homogeneous data (all same type) | Data cleaning, exploration       |
| Images, signals, scientific data | Heterogeneous data (mixed types) |

**Key Insight:** Pandas is BUILT ON NumPy. DataFrames store data as NumPy arrays internally. NumPy is the engine; Pandas is the car you drive.

## TOPIC A: Why Pandas?

### A.1 The Problem

Real-world data is **tabular** -- rows and columns with mixed types (text, numbers, dates), column headers, and it comes from CSV/Excel/databases. NumPy only gives you position-based access:

```
# NumPy: What is data[0, 2]? You have to remember.
data[0, 2] # Returns 50000 -- but what IS this?

# Pandas: Self-documenting!
df.loc['Alice', 'Salary'] # Returns 50000 -- crystal clear
```

### The NumPy Limitation (Demo)

```
import numpy as np

# With NumPy, we need separate arrays for each "column"
names = np.array(['Alice', 'Bob', 'Charlie', 'Diana'])
ages = np.array([28, 35, 42, 31])
salaries = np.array([55000, 72000, 68000, 61000])
departments = np.array(['Engineering', 'Sales', 'Engineering', 'Marketing'])

# To find all Engineers, we need boolean masking across arrays
engineer_mask = departments == 'Engineering'
print(f"Engineer names: {names[engineer_mask]}")
print(f"Engineer ages: {ages[engineer_mask]}")
print(f"Engineer salaries: {salaries[engineer_mask]}")
# Problem: 4 separate arrays for ONE table!
# Problem: We have to keep them in sync manually!
```

### The Pandas Solution

```
import pandas as pd

# All data in ONE structure
employees = pd.DataFrame({
```

```
'name': ['Alice', 'Bob', 'Charlie', 'Diana'],
'age': [28, 35, 42, 31],
'salary': [55000, 72000, 68000, 61000],
'department': ['Engineering', 'Sales', 'Engineering', 'Marketing']
})

# Finding engineers -- ONE line, clear intent
engineers = employees[employees['department'] == 'Engineering']
print(engineers)
```

```
      name  age  salary  department
0   Alice   28   55000   Engineering
2  Charlie   42   68000   Engineering
```

No sync issues. No separate arrays. Self-documenting code.

## A.2 Excel Horror Stories

**Gene Name Massacre (2016):** Excel silently corrupted gene names in genomics research. SEPT2 became "September 2", MARCH1 became "March 1". Approximately 30% of published genomics papers had corrupted data. The fix: scientists renamed actual human genes to avoid Excel corruption.

**JPMorgan London Whale (2012):** A copy-paste error in an Excel VBA macro divided by sum instead of average, understating risk by orders of magnitude. Result: **\$6.2 billion in trading losses**.

**The Pandas Alternative:** Scripted transforms, Python functions, Git version control, automated tests -- no manual copy-paste, no silent corruption.

### Excel vs Pandas

#### EXCEL:

```
+-----+
| Max ~1,048,576 rows |
| No version control  |
| Manual, click-based |
| Silent data corruption |
| Formula spaghetti   |
| Copy-paste errors   |
| "Which version?"    |
+-----+
```

#### PANDAS:

```
+-----+
| Limited by RAM only |
| Git-friendly scripts |
| Automated, scripted |
| Explicit data types |
| Readable code       |
| Reproducible pipeline |
| git log --oneline   |
+-----+
```

## A.3 NumPy vs Pandas Comparison

| Feature          | NumPy                                 | Pandas                                 |
|------------------|---------------------------------------|----------------------------------------|
| <b>Data type</b> | Homogeneous (all same type)           | Heterogeneous (mixed types per column) |
| <b>Labels</b>    | Integer positions only                | Named columns and named rows           |
| <b>Best for</b>  | Numerical computation, linear algebra | Tabular data, data cleaning, analysis  |

| Feature        | NumPy                 | Pandas                            |
|----------------|-----------------------|-----------------------------------|
| Think of it as | Scientific calculator | Spreadsheet on steroids           |
| Missing data   | No native support     | Built-in NaN handling             |
| File I/O       | Limited (.npy, .txt)  | CSV, Excel, SQL, JSON, HTML, more |

NumPy and Pandas are **partners**, not competitors.

## A.4 Import Convention

```
import pandas as pd    # Always 'pd', just like 'np' for NumPy
import numpy as np
```

Verify installation:

```
import pandas as pd
import numpy as np

print(f"Pandas version: {pd.__version__}")
print(f"NumPy version: {np.__version__}")

# Proof that Pandas is built on NumPy
s = pd.Series([1, 2, 3])
print(f"A Pandas Series stores data as: {type(s.values)}")
# Output: <class 'numpy.ndarray'>
```

## TOPIC B: Series -- 1D Labeled Array

### B.1 What is a Series?

A Series is a **1D array with labels** (called an "index"). Think of it as ONE column from a spreadsheet.

```
Index      Values
-----
Alice  ->   85
Bob    ->   92
Charlie->   78
Diana  ->   95
```

```
Name: exam_scores
dtype: int64
```

Three components: **values** (NumPy array), **index** (labels), **name** (optional column header).

## B.2 Three Ways to Create a Series

### Method 1: From a List (auto-numbered index)

```
scores = pd.Series([85, 92, 78, 95])  
print(scores)
```

```
0    85  
1    92  
2    78  
3    95  
dtype: int64
```

### Method 2: From a List with Custom Index

```
scores = pd.Series(  
    [85, 92, 78, 95],  
    index=['Alice', 'Bob', 'Charlie', 'Diana'],  
    name='exam_scores'  
)  
print(scores)
```

```
Alice    85  
Bob      92  
Charlie  78  
Diana    95  
Name: exam_scores, dtype: int64
```

### Method 3: From a Dictionary (keys become index)

```
prices = pd.Series(  
    {'Laptop': 75000,  
     'Phone': 25000,  
     'Tablet': 35000,  
     'Watch': 15000},  
    name='price_inr')  
print(prices)
```

```
Laptop    75000  
Phone     25000  
Tablet    35000  
Watch     15000  
Name: price_inr, dtype: int64
```

The dictionary keys became the index labels automatically.

## B.3 Key Series Attributes

| Attribute            | What it returns        | Example                                                  |
|----------------------|------------------------|----------------------------------------------------------|
| <code>.values</code> | Underlying NumPy array | <code>array([85, 92, 78, 95])</code>                     |
| <code>.index</code>  | The labels             | <code>Index(['Alice', 'Bob', 'Charlie', 'Diana'])</code> |
| <code>.dtype</code>  | Data type (from NumPy) | <code>int64</code>                                       |
| <code>.name</code>   | Optional name          | <code>'exam_scores'</code>                               |
| <code>.shape</code>  | Dimensions tuple       | <code>(4,)</code>                                        |
| <code>.size</code>   | Number of elements     | <code>4</code>                                           |

```
print(f"Values (NumPy array): {scores.values}")
print(f"Index: {scores.index.tolist()}")
print(f"Name: {scores.name}")
print(f>Data type: {scores.dtype}")
print(f"Shape: {scores.shape}")
print(f"Size: {scores.size}")
print(f>Type of .values: {type(scores.values)}")
```

```
Values (NumPy array): [85 92 78 95]
Index: ['Alice', 'Bob', 'Charlie', 'Diana']
Name: exam_scores
Data type: int64
Shape: (4,)
Size: 4
Type of .values: <class 'numpy.ndarray'>
```

## B.4 Accessing Series Elements

```
# By label (the Pandas way)
scores['Alice']      # 85
scores['Bob']        # 92

# By position using .iloc
scores.iloc[0]       # 85 (first element)
scores.iloc[-1]      # 95 (last element)

# Multiple labels at once (pass a list)
scores[['Alice', 'Charlie']]
# Alice      85
# Charlie    78
# Name: exam_scores, dtype: int64
```

## B.5 Series Operations (Just Like NumPy!)

```
# Element-wise arithmetic
curved_scores = scores + 5
# Alice      90
# Bob        97
# Charlie    83
# Diana     100

# Aggregations
scores.mean()    # 87.5
scores.max()     # 95
scores.min()     # 78
scores.sum()     # 350
scores.std()     # 7.59

# Boolean masking (labels preserved!)
passed = scores[scores >= 80]
# Alice      85
# Bob        92
# Diana      95

# The upgrade over NumPy
scores.idxmax()  # 'Diana' (label of max, not just position)
```

**Key insight:** Everything you know about NumPy still works. The upgrade: results KEEP THE LABELS.

## B.6 Series vs NumPy Array -- Direct Comparison

```
import numpy as np
import pandas as pd

np_scores = np.array([85, 92, 78, 95])
pd_scores = pd.Series([85, 92, 78, 95],
                      index=['Alice', 'Bob', 'Charlie', 'Diana'])

# Finding the maximum
print(f"NumPy max: {np_scores.max()} -- WHO? Position {np_scores.argmax()} (who is that?)")
print(f"Pandas max: {pd_scores.max()} -- WHO? {pd_scores.idxmax()} (clear!)")

# Filtering scores >= 80
print(f"NumPy result: {np_scores[np_scores >= 80]}")
# [85 92 95] -- just numbers, who are these people?

print("Pandas result:")
print(pd_scores[pd_scores >= 80])
# Alice      85
# Bob        92
# Diana      95
# Names AND scores -- self-documenting!
```

## TOPIC C: DataFrame Creation



## C.1 What is a DataFrame?

A DataFrame is a **2D labeled data structure** -- like a table or spreadsheet. Each **column** is a Series; all columns share the **same index**.

```

      <-- .columns -->
      name      age      salary
+----+ +-----+ +-----+ +-----+
| 0  | | Alice | | 28  | | 55000 |
.index| 1  | | Bob  | | 35  | | 72000 |
| 2  | | Charlie| | 42  | | 68000 |
+----+ +-----+ +-----+ +-----+

+----- .values -----+
      (2D NumPy array under the hood)

```

## C.2 Key DataFrame Attributes

| Attribute             | What it returns              | Example                                              |
|-----------------------|------------------------------|------------------------------------------------------|
| <code>.columns</code> | Column labels                | <code>Index(['name', 'age', 'salary'])</code>        |
| <code>.index</code>   | Row labels                   | <code>RangeIndex(start=0, stop=3, step=1)</code>     |
| <code>.values</code>  | 2D NumPy array               | <code>array([[ 'Alice', 28, 55000], ...])</code>     |
| <code>.shape</code>   | (rows, columns) tuple        | <code>(3, 3)</code>                                  |
| <code>.dtypes</code>  | Type of EACH column          | <code>name: object, age: int64, salary: int64</code> |
| <code>.size</code>    | Total elements (rows x cols) | <code>9</code>                                       |
| <code>.ndim</code>    | Number of dimensions         | <code>2</code>                                       |

**Note:** `.dtype` (singular) is for Series; `.dtypes` (plural) is for DataFrame.

## C.3 Method 1: From a Dictionary (Most Common)

Dictionary keys become column names; values (lists) become column data.

```

employees = pd.DataFrame({
    'name':      ['Alice', 'Bob', 'Charlie', 'Diana'],
    'age':       [28, 35, 42, 31],
    'salary':    [55000, 72000, 68000, 61000],
    'department': ['Engineering', 'Marketing', 'Engineering', 'Sales']
})
print(employees)

```

```

      name  age  salary  department
0   Alice   28   55000  Engineering
1    Bob   35   72000    Marketing

```

|   |         |    |       |             |
|---|---------|----|-------|-------------|
| 2 | Charlie | 42 | 68000 | Engineering |
| 3 | Diana   | 31 | 61000 | Sales       |

```
print(f"Shape: {employees.shape}")      # (4, 4)
print(f"Columns: {employees.columns.tolist()}")
# ['name', 'age', 'salary', 'department']
print(employees.dtypes)
# name          object
# age           int64
# salary        int64
# department    object
```

**Rule:** All lists must have the **same length** (same number of rows).

## C.4 Method 2: From a List of Lists

Each inner list is one ROW. You MUST provide column names with `columns=`.

```
data_rows = [
    ['Alice', 28, 55000, 'Engineering'],
    ['Bob', 35, 72000, 'Marketing'],
    ['Charlie', 42, 68000, 'Engineering'],
    ['Diana', 31, 61000, 'Sales']
]
df = pd.DataFrame(data_rows, columns=['name', 'age', 'salary', 'department'])
```

Without `columns=`, columns get named 0, 1, 2, 3 -- meaningless.

## C.5 Method 3: From a NumPy Array

Add labels to existing NumPy data.

```
import numpy as np

arr = np.array([
    [85, 92, 78],
    [90, 88, 95],
    [76, 81, 89],
    [93, 87, 91]
])

grades = pd.DataFrame(
    arr,
    columns=['Math', 'Science', 'English'],
    index=['Alice', 'Bob', 'Charlie', 'Diana']
)
print(grades)
```

|       | Math | Science | English |
|-------|------|---------|---------|
| Alice | 85   | 92      | 78      |

|         |    |    |    |
|---------|----|----|----|
| Bob     | 90 | 88 | 95 |
| Charlie | 76 | 81 | 89 |
| Diana   | 93 | 87 | 91 |

```
print(f"df.values returns: {type(grades.values)}")
# <class 'numpy.ndarray'>
# Proof: Pandas is a LABEL LAYER on top of NumPy!
```

## C.6 Method 4: From a List of Dictionaries (API/JSON Style)

Each dictionary is one row; keys are column names.

```
records = [
    {'name': 'Alice', 'age': 28, 'salary': 55000},
    {'name': 'Bob', 'age': 35, 'salary': 72000},
    {'name': 'Charlie', 'age': 42, 'salary': 68000}
]
df = pd.DataFrame(records)
```

**Missing keys become NaN:**

```
records_incomplete = [
    {'name': 'Alice', 'age': 28, 'salary': 55000},
    {'name': 'Bob', 'age': 35}, # no 'salary'!
]
df = pd.DataFrame(records_incomplete)
#   name  age  salary
# 0  Alice  28.0  55000.0
# 1   Bob   35.0     NaN  <-- NaN = missing value
```

**Note:** NaN is a float, so the salary column gets upcasted from int64 to float64.

## C.7 Creation Methods Summary

| Method        | Input                                             | Think of data as...     | columns=<br>needed?   |
|---------------|---------------------------------------------------|-------------------------|-----------------------|
| Dict of lists | <code>{ 'A': [1,2], 'B': [3,4] }</code>           | Column-by-column        | No (keys are columns) |
| List of lists | <code>[[1,3], [2,4]]</code>                       | Row-by-row              | Yes                   |
| NumPy array   | <code>np.array([[1,3], [2,4]])</code>             | Add labels to numbers   | Yes                   |
| List of dicts | <code>[{ 'A':1, 'B':3 }, { 'A':2, 'B':4 }]</code> | One dict per row (JSON) | No (keys are columns) |

# TOPIC D: Reading External Data + Exploration

## D.1 Common Data Formats

| Format                       | File Extension | Pandas Function   |
|------------------------------|----------------|-------------------|
| CSV (Comma-Separated Values) | .csv           | pd.read_csv()     |
| Excel                        | .xlsx          | pd.read_excel()   |
| JSON                         | .json          | pd.read_json()    |
| SQL Database                 | --             | pd.read_sql()     |
| Parquet (big data)           | .parquet       | pd.read_parquet() |

The most common is CSV -- simple text files where values are separated by commas.

## D.2 pd.read\_csv() -- Your Data Loading Workhorse

Basic usage:

```
df = pd.read_csv('filename.csv')
```

### Common Parameters

| Parameter | What It Does                     | Example                 | When to Use                            |
|-----------|----------------------------------|-------------------------|----------------------------------------|
| sep       | Column separator                 | sep='\t'                | Tab-separated files                    |
| header    | Row with column names            | header=0 (default)      | Headers on a different row             |
| index_col | Column to use as row index       | index_col='id'          | Column is a unique identifier          |
| usecols   | Only load specific columns       | usecols=['name', 'age'] | File has 100 columns, need 3           |
| nrows     | Only load first N rows           | nrows=100               | Quick preview of huge files            |
| dtype     | Force column data types          | dtype={'zip': str}      | Prevent IDs/zips from becoming numbers |
| na_values | Extra values to treat as missing | na_values=['N/A', '-']  | Custom missing data markers            |

### Key Parameters Demo

```

import pandas as pd
import io

csv_data = """order_id,customer_name,product,category,price,quantity,city
ORD001,Priya Sharma,Wireless Mouse,Electronics,899,2,Mumbai
ORD002,Rahul Verma,Python Crash Course,Books,599,1,Delhi
ORD003,Anita Desai,USB-C Hub,Electronics,1299,1,Bangalore"""

# Basic load
df = pd.read_csv(io.StringIO(csv_data))

# Load only specific columns
df_subset = pd.read_csv(io.StringIO(csv_data), usecols=['customer_name', 'product', 'price'])

# Load only first N rows
df_preview = pd.read_csv(io.StringIO(csv_data), nrows=2)

# Force column types (prevent leading zeros from being dropped)
csv_ids = """id,name,zip_code
001,Alice,110001
002,Bob,400001"""

df_typed = pd.read_csv(io.StringIO(csv_ids), dtype={'id': str, 'zip_code': str})
# '001' stays as '001' instead of becoming 1

```

**Pro tip:** ALWAYS use `dtype=str` for IDs, zip codes, and phone numbers.

## D.3 The Five-Method Exploration Ritual

Run these 5 methods **every time** you load a dataset. No exceptions.

```

STEP 1: df.head()      -> "What does the data look like?"    (first 5 rows)
STEP 2: df.tail()     -> "Any weird stuff at the end?"      (last 5 rows)
STEP 3: df.shape       -> "How big is it?"                  (rows, columns)
STEP 4: df.info()      -> "What types? Any missing values?" (data health check)
STEP 5: df.describe() -> "What are the statistics?"         (numeric summary)

```

### Example

```

df.head()      # First 5 rows
df.tail()      # Last 5 rows

df.shape       # (10, 7) -- NOTE: attribute, not method! No parentheses.
# 10 rows, 7 columns

df.info()
# <class 'pandas.core.frame.DataFrame'>
# RangeIndex: 10 entries, 0 to 9
# Data columns (total 7 columns):
# #   Column      Non-Null Count  Dtype
# ---  -
# 0   order_id    10 non-null    object
# 1   customer_name 10 non-null    object

```

```
# 2  product      10 non-null    object
# 3  category     10 non-null    object
# 4  price        10 non-null    int64
# 5  quantity     10 non-null    int64
# 6  city         10 non-null    object
```

```
df.describe()
#               price  quantity
# count    10.000000  10.000000
# mean     1329.000000  1.500000
# std       935.776...  1.080...
# min       349.000000  1.000000
# 25%       636.500000  1.000000
# 50%       899.000000  1.000000
# 75%      2124.000000  2.000000
# max      3499.000000  4.000000
```

## Reading .info() Output

How to detect missing values:  
total rows - non-null count = missing values

Example:  
RangeIndex: 500 entries  
price 478 non-null float64  
--> 500 - 478 = 22 MISSING values in 'price'

**Red flags:** Non-null count < total rows = MISSING DATA. dtype = `object` for a column that should be numeric = data quality issue.

## describe(include='all')

```
df.describe(include='all')
# Adds text column statistics:
# unique = number of distinct values
# top    = most frequent value
# freq   = count of most frequent value
```

## D.4 The Session Dataset

This dataset is used for all examples in Topics E through H:

```
import pandas as pd
import numpy as np

orders = pd.DataFrame({
    'order_id': ['ORD001', 'ORD002', 'ORD003', 'ORD004', 'ORD005',
                'ORD006', 'ORD007', 'ORD008', 'ORD009', 'ORD010',
                'ORD011', 'ORD012', 'ORD013', 'ORD014', 'ORD015'],
    'customer_name': ['Priya Sharma', 'Rahul Verma', 'Anita Desai',
                     'Vikram Patel', 'Sneha Iyer', 'Arjun Nair',
```

```

        'Meera Joshi', 'Karan Singh', 'Divya Rao',
        'Amit Gupta', 'Priya Sharma', 'Neha Kulkarni',
        'Rohan Mehta', 'Sneha Iyer', 'Kavita Reddy'],
    'product': ['Wireless Mouse', 'Python Crash Course', 'USB-C Hub',
               'Cotton T-Shirt', 'Desk Lamp', 'Data Science Handbook',
               'Bluetooth Speaker', 'Running Shoes', 'Notebook Set',
               'Mechanical Keyboard', 'Phone Case', 'Yoga Mat',
               'LED Monitor', 'Wall Clock', 'Backpack'],
    'category': ['Electronics', 'Books', 'Electronics', 'Clothing', 'Home',
                 'Books', 'Electronics', 'Clothing', 'Home', 'Electronics',
                 'Electronics', 'Clothing', 'Electronics', 'Home', 'Clothing'],
    'price': [899, 599, 1299, 499, 749, 899, 1999, 2499, 349, 3499,
              199, 1299, 8999, 599, 1799],
    'quantity': [2, 1, 1, 3, 1, 2, 1, 1, 4, 1, 2, 1, 1, 2, 1],
    'city': ['Mumbai', 'Delhi', 'Bangalore', 'Chennai', 'Mumbai',
             'Kolkata', 'Pune', 'Delhi', 'Bangalore', 'Mumbai',
             'Chennai', 'Bangalore', 'Delhi', 'Mumbai', 'Kolkata'],
    'order_date': ['2024-01-15', '2024-01-15', '2024-01-16', '2024-01-16',
                   '2024-01-17', '2024-01-17', '2024-01-18', '2024-01-18',
                   '2024-01-19', '2024-01-19', '2024-01-20', '2024-01-20',
                   '2024-01-21', '2024-01-21', '2024-01-22']
})

```

### Key properties:

- 15 rows, 8 columns
- Categories: Electronics (6), Clothing (4), Home (3), Books (2)
- Cities: Mumbai (4), Delhi (3), Bangalore (3), Chennai (2), Kolkata (2), Pune (1)
- Price range: 199 to 8999
- Repeat customers: Priya Sharma (rows 0, 10), Sneha Iyer (rows 4, 13)

```

print(f"Shape: {orders.shape}")    # (15, 8)
print(f"Columns: {orders.columns.tolist()}")
# ['order_id', 'customer_name', 'product', 'category', 'price', 'quantity', 'city', 'orde

```

## TOPIC E: Column Selection

### E.1 Single Column Selection

Two syntaxes, both return a **Series**:

```

# Method 1: Bracket notation (ALWAYS works)
orders['product']          # Returns a Series

# Method 2: Dot notation (convenient shortcut)
orders.product            # Also returns a Series

```

**When Dot Notation FAILS (3 cases):**

| Scenario                     | Dot Notation                                                | Bracket Notation                                |
|------------------------------|-------------------------------------------------------------|-------------------------------------------------|
| Column name has spaces       | <code>df.order date</code> -- <b>SyntaxError</b>            | <code>df['order date']</code> -- works          |
| Column name matches a method | <code>df.count</code> -- returns the <b>method!</b>         | <code>df['count']</code> -- returns column data |
| Column name in a variable    | <code>col = 'price'; df.col</code> -- <b>AttributeError</b> | <code>df[col]</code> -- works                   |

**Rule: Always use bracket notation.** Dot notation is a shortcut that can silently break.

## E.2 Multiple Column Selection

Pass a **list** of column names inside brackets. Returns a **DataFrame**:

```
subset = orders[['customer_name', 'product', 'price']]
print(type(subset))    # <class 'pandas.core.frame.DataFrame'>
print(subset.shape)    # (15, 3)
```

## E.3 The Critical Distinction: `df['col']` vs `df[['col']]`

```
series_result = orders['price']    # Series (1D)
df_result     = orders[['price']]  # DataFrame (2D)

print(type(series_result))    # pandas.core.series.Series
print(series_result.shape)    # (15,)

print(type(df_result))        # pandas.core.frame.DataFrame
print(df_result.shape)        # (15, 1)
```

**Same data, different container:**

|                                                                                                                                |                                                                                                                    |
|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <pre>orders['price'] ----- 0      899 1      599 2     1299 3      499 ... Name: price dtype: int64 (Series - no header)</pre> | <pre>orders[['price']] -----       price 0      899 1      599 2     1299 ... (DataFrame with column header)</pre> |
|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|

## E.4 Column Selection Cheat Sheet



| Syntax                            | Returns   | Result Type      | Use When                                      |
|-----------------------------------|-----------|------------------|-----------------------------------------------|
| <code>df['col']</code>            | Series    | 1D, shape (n,)   | Calculations: <code>df['price'].mean()</code> |
| <code>df.col</code>               | Series    | 1D, shape (n,)   | Quick exploration (simple names only)         |
| <code>df[['col1', 'col2']]</code> | DataFrame | 2D, shape (n, 2) | Sub-tables for display/export                 |
| <code>df[['col']]</code>          | DataFrame | 2D, shape (n, 1) | When you need a table, not a column           |

## TOPIC F: Row Selection -- .loc vs .iloc (CRITICAL)

### F.1 The Two Ways to Select Rows

`.loc[]` -> Select by LABEL (name/index value)  
The 'L' in loc = Label

`.iloc[]` -> Select by INTEGER POSITION (0, 1, 2, ...)  
The 'i' in iloc = Integer

#### Memory Tricks:

- `.loc` -- loc = Label
- `.iloc` -- iloc = Integer position
- Both use **square brackets**, not parentheses: `.loc[]` not `.loc()`

### F.2 .loc[] -- Label-Based Selection

| Syntax                                   | Meaning                   | Example                                          |
|------------------------------------------|---------------------------|--------------------------------------------------|
| <code>df.loc[label]</code>               | Single row                | <code>orders.loc[3]</code>                       |
| <code>df.loc[label, 'col']</code>        | Specific cell             | <code>orders.loc[3, 'product']</code>            |
| <code>df.loc[start:end]</code>           | Row slice<br>(INCLUSIVE!) | <code>orders.loc[0:3]</code> -- 4 rows           |
| <code>df.loc[:, 'col']</code>            | All rows, one column      | <code>orders.loc[:, 'price']</code>              |
| <code>df.loc[label, ['c1', 'c2']]</code> | One row, multiple cols    | <code>orders.loc[3, ['product', 'price']]</code> |

```
# Single row by label
orders.loc[3]
# order_id          ORD004
# customer_name     Vikram Patel
# product           Cotton T-Shirt
# ...
# Name: 3, dtype: object    <-- Returns a Series

# Specific cell
orders.loc[3, 'product']
# 'Cotton T-Shirt'          <-- Returns a scalar

# Row slice -- INCLUSIVE on both ends!
orders.loc[0:3]
# Returns 4 rows: labels 0, 1, 2, AND 3

# Row + column selection
orders.loc[0:2, ['customer_name', 'product', 'price']]
#   customer_name          product  price
# 0  Priya Sharma      Wireless Mouse    899
# 1   Rahul Verma  Python Crash Course    599
# 2   Anita Desai           USB-C Hub   1299
```

### F.3 .iloc[] -- Integer Position Selection

| Syntax                             | Meaning                | Example                                               |
|------------------------------------|------------------------|-------------------------------------------------------|
| <code>df.iloc[pos]</code>          | Single row             | <code>df.iloc[0]</code> -- first row                  |
| <code>df.iloc[pos, col_pos]</code> | Specific cell          | <code>df.iloc[0, 2]</code> -- first row, third column |
| <code>df.iloc[start:end]</code>    | Row slice (EXCLUSIVE!) | <code>df.iloc[0:3]</code> -- 3 rows                   |
| <code>df.iloc[:, start:end]</code> | All rows, col slice    | <code>df.iloc[:, 0:3]</code> -- first 3 columns       |
| <code>df.iloc[-1]</code>           | Last row               | Negative indexing works!                              |

```
# First row
orders.iloc[0]

# Last row (negative indexing)
orders.iloc[-1]

# Specific cell by position
orders.iloc[0, 2]    # First row, third column
# 'Wireless Mouse'

# Row slice -- EXCLUSIVE end (standard Python)
orders.iloc[0:3]
# Returns 3 rows: positions 0, 1, 2

# Every other row
orders.iloc[::2]      # Rows at positions 0, 2, 4, 6, ...

# Row + column by position
```

```
orders.iloc[0:2, 0:3]
#  order_id  customer_name      product
# 0   ORD001    Priya Sharma  Wireless Mouse
# 1   ORD002    Rahul Verma   Python Crash Course
```

## F.4 The Critical Comparison Table

| Feature              | .loc[]                      | .iloc[]                                 |
|----------------------|-----------------------------|-----------------------------------------|
| Selects by           | Labels (names/index values) | Integer positions (0, 1, 2, ...)        |
| End of slice         | <b>INCLUSIVE</b>            | <b>EXCLUSIVE</b> (like Python lists)    |
| Memory trick         | loc = Label                 | iloc = Integer position                 |
| [0:3] returns        | 4 rows (labels 0, 1, 2, 3)  | 3 rows (positions 0, 1, 2)              |
| Negative indexing    | Not common                  | Works: <code>iloc[-1]</code> = last row |
| Column specification | By name: 'price'            | By position: 2                          |
| When to use          | Know the label/name         | Know the position                       |

### The Slicing Trap Visualized

```
Row index:  0      1      2      3      4      5
            |      |      |      |      |      |

loc[0:3]:    [-----|-----|-----|-----]    <-- 4 items (INCLUSIVE)
iloc[0:3]:    [-----|-----|-----]              <-- 3 items (EXCLUSIVE)
                    ^
                loc INCLUDES this
                iloc EXCLUDES this
```

### Slicing Comparison Demo

```
# .loc slicing (INCLUSIVE end)
loc_result = orders.loc[0:3]
print(f"orders.loc[0:3] -> {len(loc_result)} rows")    # 4 rows

# .iloc slicing (EXCLUSIVE end)
iloc_result = orders.iloc[0:3]
print(f"orders.iloc[0:3] -> {len(iloc_result)} rows")  # 3 rows

# DIFFERENCE: 1 extra row with .loc!
```

## F.5 Custom Index Changes Everything

When the index is NOT default integers, the difference becomes stark:

```
products = pd.DataFrame({
    'product': ['Wireless Mouse', 'USB-C Hub', 'Mechanical Keyboard'],
    'price': [899, 1299, 3499],
}, index=['WM01', 'UH02', 'MK03'])

# .loc uses LABELS (string codes)
products.loc['MK03']
# product    Mechanical Keyboard
# price              3499
# Name: MK03, dtype: object

# .iloc uses POSITIONS (still integers)
products.iloc[2]
# Same row! But accessed by different logic.

# products.loc[2] --> KeyError! No label called 2
# products.iloc['MK03'] --> TypeError! iloc only accepts integers
```

## F.6 Row + Column Selection Combined

```
# .loc: rows by LABEL, columns by NAME (more readable)
loc_combined = orders.loc[0:2, ['customer_name', 'product', 'price']]
print(loc_combined)
```

|   | customer_name | product             | price |
|---|---------------|---------------------|-------|
| 0 | Priya Sharma  | Wireless Mouse      | 899   |
| 1 | Rahul Verma   | Python Crash Course | 599   |
| 2 | Anita Desai   | USB-C Hub           | 1299  |

Shape: (3, 3) -- 3 rows (labels 0,1,2 inclusive), 3 columns by name.

```
# .iloc: rows by POSITION, columns by POSITION
iloc_combined = orders.iloc[0:2, 0:3]
print(iloc_combined)
```

|   | order_id | customer_name | product             |
|---|----------|---------------|---------------------|
| 0 | ORD001   | Priya Sharma  | Wireless Mouse      |
| 1 | ORD002   | Rahul Verma   | Python Crash Course |

Shape: (2, 3) -- 2 rows (positions 0,1 exclusive end), 3 columns by position.

### KEY INSIGHT:

- `.loc` uses NAMES for columns -- more readable
- `.iloc` uses POSITIONS for columns -- more concise
- In practice, `.loc` is preferred because it is self-documenting.

## F.7 Chained Indexing -- The Anti-Pattern

```
# BAD -- chained indexing (may modify a copy!)
orders['product'][3]          # "Works" for reading, but...
orders['product'][3] = 'Silk Shirt' # May NOT actually modify orders!
# --> SettingWithCopyWarning

# GOOD -- direct indexing with .loc (always reliable)
orders.loc[3, 'product']      # Reading: always works
orders.loc[3, 'product'] = 'Silk Shirt' # Writing: always modifies original
```

**Rule: ALWAYS use `.loc[row, col]` for reading AND writing. NEVER use `df['col'][row]`.**

## TOPIC G: Boolean Filtering

### G.1 The Pattern

```
df[df['column'] condition]
```

#### How it works:

`df['city'] == 'Mumbai'` creates a boolean mask (True/False per row)

`df[mask]` keeps only rows where mask is True

Result is a NEW DataFrame (original unchanged)

This is the same as NumPy boolean masking from Session 3.2!

Step 1: `orders['city'] == 'Mumbai'`  
 -> `[True, False, False, False, True, False, ..., True, False]`

Step 2: `orders[mask]`  
 -> Only True rows are kept

Step 3: Result = new DataFrame with 4 Mumbai rows

### G.2 Single Condition Filtering

| Operator          | Meaning      | Example                                 |
|-------------------|--------------|-----------------------------------------|
| <code>==</code>   | Equal to     | <code>df[df['city'] == 'Mumbai']</code> |
| <code>!=</code>   | Not equal to | <code>df[df['city'] != 'Mumbai']</code> |
| <code>&gt;</code> | Greater than | <code>df[df['price'] &gt; 1000]</code>  |

| Operator           | Meaning          | Example                                 |
|--------------------|------------------|-----------------------------------------|
| <code>&gt;=</code> | Greater or equal | <code>df[df['quantity'] &gt;= 2]</code> |
| <code>&lt;</code>  | Less than        | <code>df[df['price'] &lt; 500]</code>   |
| <code>&lt;=</code> | Less or equal    | <code>df[df['price'] &lt;= 899]</code>  |

```
# Mumbai orders
mumbai_orders = orders[orders['city'] == 'Mumbai']
# orders still has 15 rows; mumbai_orders has 4 rows

# Expensive items
orders[orders['price'] > 2000]

# Not Electronics
orders[orders['category'] != 'Electronics']
```

**Common mistake:** Using `=` instead of `==` :

```
orders[orders['city'] = 'Mumbai'] # SyntaxError!
orders[orders['city'] == 'Mumbai'] # Correct!
```

## G.3 Multiple Conditions

| Operation | Symbol             | English           |
|-----------|--------------------|-------------------|
| AND       | <code>&amp;</code> | Both must be True |
| OR        | <code> </code>     | At least one True |
| NOT       | <code>~</code>     | Invert True/False |

**MANDATORY PARENTHESES around each condition!**

```
# CORRECT -- parentheses around each condition
orders[(orders['city'] == 'Mumbai') & (orders['price'] > 1000)]

# WRONG -- missing parentheses -> confusing error!
orders[orders['city'] == 'Mumbai' & orders['price'] > 1000]
# TypeError!
```

## Examples

```
# AND -- both conditions must be true
result_and = orders[(orders['city'] == 'Mumbai') & (orders['category'] == 'Electronics')]
print(result_and[['order_id', 'product', 'category', 'city', 'price']])
```

|   | order_id | product             | category    | city   | price |
|---|----------|---------------------|-------------|--------|-------|
| 0 | ORD001   | Wireless Mouse      | Electronics | Mumbai | 899   |
| 9 | ORD010   | Mechanical Keyboard | Electronics | Mumbai | 3499  |

```
# OR -- either condition
result_or = orders[(orders['city'] == 'Mumbai') | (orders['city'] == 'Delhi')]
print(result_or[['order_id', 'product', 'city', 'price']])
```

|    | order_id | product             | city   | price |
|----|----------|---------------------|--------|-------|
| 0  | ORD001   | Wireless Mouse      | Mumbai | 899   |
| 1  | ORD002   | Python Crash Course | Delhi  | 599   |
| 4  | ORD005   | Desk Lamp           | Mumbai | 749   |
| 7  | ORD008   | Running Shoes       | Delhi  | 2499  |
| 9  | ORD010   | Mechanical Keyboard | Mumbai | 3499  |
| 12 | ORD013   | LED Monitor         | Delhi  | 8999  |
| 13 | ORD014   | Wall Clock          | Mumbai | 599   |

```
# NOT -- invert
orders[~(orders['category'] == 'Books')]
# Returns 13 rows (everything except the 2 Books orders)

# Complex combination
complex_filter = orders[
    (orders['category'] == 'Electronics') &
    (orders['price'] > 1000) &
    ((orders['city'] == 'Mumbai') | (orders['city'] == 'Delhi'))
]
print(complex_filter[['order_id', 'product', 'price', 'city']])
```

|    | order_id | product             | price | city   |
|----|----------|---------------------|-------|--------|
| 9  | ORD010   | Mechanical Keyboard | 3499  | Mumbai |
| 12 | ORD013   | LED Monitor         | 8999  | Delhi  |

## G.4 Boolean Filtering Under the Hood

To truly understand filtering, visualize the mask creation step:

```
# Step 1: Create the mask (a Series of True/False)
mask = orders['city'] == 'Mumbai'
print(mask)
```

|   |       |
|---|-------|
| 0 | True  |
| 1 | False |
| 2 | False |
| 3 | False |
| 4 | True  |
| 5 | False |

```

6      False
7      False
8      False
9      True
10     False
11     False
12     False
13     True
14     False
Name: city, dtype: bool

```

```

# Step 2: Count how many match
print(f"Matching rows: {mask.sum()} out of {len(mask)}")
# Matching rows: 4 out of 15

# Step 3: Apply the mask -- only True rows are kept
mumbai_orders = orders[mask]
print(mumbai_orders[['order_id', 'customer_name', 'product', 'city', 'price']])

```

|    | order_id | customer_name | product             | city   | price |
|----|----------|---------------|---------------------|--------|-------|
| 0  | ORD001   | Priya Sharma  | Wireless Mouse      | Mumbai | 899   |
| 4  | ORD005   | Sneha Iyer    | Desk Lamp           | Mumbai | 749   |
| 9  | ORD010   | Amit Gupta    | Mechanical Keyboard | Mumbai | 3499  |
| 13 | ORD014   | Sneha Iyer    | Wall Clock          | Mumbai | 599   |

**The process is always the same:** condition creates boolean mask, mask selects rows.

## G.5 .query() Method -- Cleaner Syntax

For complex filters, `.query()` is more readable (no need to repeat `orders[...]` for every condition):

```

# Standard boolean filtering (verbose)
orders[(orders['city'] == 'Mumbai') & (orders['price'] > 1000)]

# Equivalent .query() syntax (clean!)
orders.query("city == 'Mumbai' and price > 1000")

```

### .query() Syntax Differences

| Feature      | Standard Boolean                   | .query()                                |
|--------------|------------------------------------|-----------------------------------------|
| AND          | <code>&amp;</code>                 | <code>and</code>                        |
| OR           | <code> </code>                     | <code>or</code>                         |
| NOT          | <code>~</code>                     | <code>not</code>                        |
| Parentheses  | Mandatory around each condition    | Only when needed for grouping           |
| Column names | <code>df['column']</code> repeated | Just write <code>column</code> directly |



## String Quoting in .query()

Outer quotes: double `"` . Inner string values: single `'` .

```
orders.query("city == 'Mumbai'")
orders.query("category == 'Electronics' and price > 1000")
```

## Using Variables with @ in .query()

```
min_price = 2000
target_cities = ['Mumbai', 'Delhi']

result = orders.query("price > @min_price and city in @target_cities")
print(result[['order_id', 'product', 'price', 'city']])
```

|    | order_id | product             | price | city   |
|----|----------|---------------------|-------|--------|
| 7  | ORD008   | Running Shoes       | 2499  | Delhi  |
| 9  | ORD010   | Mechanical Keyboard | 3499  | Mumbai |
| 12 | ORD013   | LED Monitor         | 8999  | Delhi  |

The `@` prefix tells `.query()` to use a Python variable, not a column name.

# TOPIC H: Sorting

## H.1 .sort\_values() -- Ordering Your Results

```
# Ascending (default) -- cheapest first
orders.sort_values('price')

# Descending -- most expensive first
orders.sort_values('price', ascending=False)

# Multi-column sort (tie-breaking)
orders.sort_values(['category', 'price'], ascending=[True, False])
# Category A-Z, then within each category: price high-to-low
```

## Parameters

| Parameter              | Default    | Meaning                                       |
|------------------------|------------|-----------------------------------------------|
| <code>by</code>        | (required) | Column name or list of names                  |
| <code>ascending</code> | True       | True = A-Z / low-high, False = Z-A / high-low |

| Parameter   | Default | Meaning                              |
|-------------|---------|--------------------------------------|
| na_position | 'last'  | Where to put NaN values              |
| inplace     | False   | If True, modifies DataFrame directly |

**Key behavior:** Returns a NEW DataFrame. The original is unchanged. Assign the result!

```
sorted_orders = orders.sort_values('price', ascending=False)
# 'orders' is still in original order
# 'sorted_orders' is sorted
```

## Ascending Sort Example

```
sorted_asc = orders.sort_values('price')
print(sorted_asc[['order_id', 'product', 'price']].head(5))
```

|    | order_id | product             | price |
|----|----------|---------------------|-------|
| 10 | ORD011   | Phone Case          | 199   |
| 8  | ORD009   | Notebook Set        | 349   |
| 3  | ORD004   | Cotton T-Shirt      | 499   |
| 1  | ORD002   | Python Crash Course | 599   |
| 13 | ORD014   | Wall Clock          | 599   |

## Descending Sort Example

```
sorted_desc = orders.sort_values('price', ascending=False)
print(sorted_desc[['order_id', 'product', 'price']].head(5))
```

|    | order_id | product             | price |
|----|----------|---------------------|-------|
| 12 | ORD013   | LED Monitor         | 8999  |
| 9  | ORD010   | Mechanical Keyboard | 3499  |
| 7  | ORD008   | Running Shoes       | 2499  |
| 6  | ORD007   | Bluetooth Speaker   | 1999  |
| 14 | ORD015   | Backpack            | 1799  |

## Multi-Column Sort Example

```
sorted_multi = orders.sort_values(['category', 'price'], ascending=[True, False])
print(sorted_multi[['order_id', 'product', 'category', 'price']])
```

|   | order_id | product               | category | price |
|---|----------|-----------------------|----------|-------|
| 5 | ORD006   | Data Science Handbook | Books    | 899   |
| 1 | ORD002   | Python Crash Course   | Books    | 599   |
| 7 | ORD008   | Running Shoes         | Clothing | 2499  |

|    |        |                     |             |      |
|----|--------|---------------------|-------------|------|
| 14 | ORD015 | Backpack            | Clothing    | 1799 |
| 11 | ORD012 | Yoga Mat            | Clothing    | 1299 |
| 3  | ORD004 | Cotton T-Shirt      | Clothing    | 499  |
| 12 | ORD013 | LED Monitor         | Electronics | 8999 |
| 9  | ORD010 | Mechanical Keyboard | Electronics | 3499 |
| 6  | ORD007 | Bluetooth Speaker   | Electronics | 1999 |
| 2  | ORD003 | USB-C Hub           | Electronics | 1299 |
| 0  | ORD001 | Wireless Mouse      | Electronics | 899  |
| 10 | ORD011 | Phone Case          | Electronics | 199  |
| 4  | ORD005 | Desk Lamp           | Home        | 749  |
| 13 | ORD014 | Wall Clock          | Home        | 599  |
| 8  | ORD009 | Notebook Set        | Home        | 349  |

Within each category, products are sorted by price from high to low.

## H.2 .sort\_index() and .reset\_index()

After filtering, your index has "holes" (e.g., [0, 4, 9, 13]). After sorting, your index is shuffled.

```
# .sort_index() -- restore original row order
filtered_data.sort_index()

# .reset_index(drop=True) -- create clean 0, 1, 2, ... index
filtered_data.reset_index(drop=True)
```

**Why drop=True ?** Without it, the old index becomes a new column (usually not what you want).

| BEFORE reset_index: |          |       | AFTER reset_index(drop=True): |          |       |
|---------------------|----------|-------|-------------------------------|----------|-------|
|                     | order_id | price |                               | order_id | price |
| 0                   | ORD001   | 899   | 0                             | ORD001   | 899   |
| 4                   | ORD005   | 749   | 1                             | ORD005   | 749   |
| 9                   | ORD010   | 3499  | 2                             | ORD010   | 3499  |
| 13                  | ORD014   | 599   | 3                             | ORD014   | 599   |

## H.3 The Clean Pipeline

```
# Filter -> Sort -> Reset Index
result = (orders[orders['city'] == 'Mumbai']
          .sort_values('price', ascending=False)
          .reset_index(drop=True))
print(result[['order_id', 'customer_name', 'product', 'price']])
```

|   | order_id | customer_name | product             | price |
|---|----------|---------------|---------------------|-------|
| 0 | ORD010   | Amit Gupta    | Mechanical Keyboard | 3499  |
| 1 | ORD001   | Priya Sharma  | Wireless Mouse      | 899   |
| 2 | ORD005   | Sneha Iyer    | Desk Lamp           | 749   |
| 3 | ORD014   | Sneha Iyer    | Wall Clock          | 599   |

This reads like English: "Take orders, keep Mumbai, sort by price descending, reset index."

# KEY FRAMEWORKS & QUICK REFERENCES

## NumPy vs Pandas Decision Table

| Situation                            | Use NumPy | Use Pandas |
|--------------------------------------|-----------|------------|
| Pure math on numbers                 | Yes       | No         |
| Tabular data with column names       | No        | Yes        |
| Data from CSV/Excel files            | No        | Yes        |
| Matrix operations                    | Yes       | No         |
| Mixed types (text + numbers)         | No        | Yes        |
| Boolean masking on labeled data      | Could     | Preferred  |
| Data cleaning and exploration        | No        | Yes        |
| Performance-critical numerical loops | Yes       | No         |

## loc vs iloc Comparison Table (THE Most Important Table)

|                | .loc[]                      | .iloc[]                           |
|----------------|-----------------------------|-----------------------------------|
| Stands for     | Label-based location        | Integer-position location         |
| Selects by     | Index labels/names          | Integer positions (0, 1, 2, ...)  |
| Slice end      | INCLUSIVE                   | EXCLUSIVE                         |
| [0:3] result   | 4 rows (0, 1, 2, 3)         | 3 rows (0, 1, 2)                  |
| Column access  | By name: 'price'            | By position: 4                    |
| Negative index | Not typical                 | Works: -1 = last                  |
| Best for       | Named lookups, label ranges | First/last N rows, positional ops |
| Analogy        | Find book by title          | Find book by shelf position       |

## Column Selection Cheat Sheet

| What You Write | What You Get          | Result Type | Shape Example |
|----------------|-----------------------|-------------|---------------|
| df['price']    | One column            | Series      | (15,)         |
| df.price       | One column (shortcut) | Series      | (15,)         |

| What You Write                    | What You Get          | Result Type      | Shape Example        |
|-----------------------------------|-----------------------|------------------|----------------------|
| <code>df[['price']]</code>        | One column as a table | <b>DataFrame</b> | <code>(15, 1)</code> |
| <code>df[['price', 'qty']]</code> | Multiple columns      | <b>DataFrame</b> | <code>(15, 2)</code> |

## Boolean Operators: Python vs Pandas

| Operation | Python (scalars) | Pandas (Series)    | .query()         |
|-----------|------------------|--------------------|------------------|
| AND       | <code>and</code> | <code>&amp;</code> | <code>and</code> |
| OR        | <code>or</code>  | <code> </code>     | <code>or</code>  |
| NOT       | <code>not</code> | <code>~</code>     | <code>not</code> |

**In Pandas filters:** Use `&`, `|`, `~` with MANDATORY parentheses around each condition.

## COMMON ERRORS QUICK REFERENCE

| # | Error                                                         | Cause                                                                                  | Fix                                                                                    |
|---|---------------------------------------------------------------|----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| 1 | <code>SyntaxError</code>                                      | Used <code>=</code> instead of <code>==</code> in filter                               | <code>df[df['city'] == 'Mumbai']</code> (double equals)                                |
| 2 | <code>TypeError: Cannot perform 'rand_...'</code>             | Missing parentheses in multi-condition filter                                          | Wrap each condition in <code>()</code> : <code>(cond1) &amp; (cond2)</code>            |
| 3 | <code>ValueError: truth value of a Series is ambiguous</code> | Used <code>and</code> / <code>or</code> instead of <code>&amp;</code> / <code> </code> | Replace <code>and</code> with <code>&amp;</code> , <code>or</code> with <code> </code> |
| 4 | Off-by-one in slicing                                         | Confused <code>.loc</code> (inclusive) with <code>.iloc</code> (exclusive)             | <code>.loc[0:3]</code> = 4 rows;<br><code>.iloc[0:3]</code> = 3 rows                   |
| 5 | <code>SyntaxError</code> or wrong result with dot notation    | Column name has space, or matches a method name                                        | Always use <code>df['column']</code> bracket notation                                  |
| 6 | Unwanted "index" column after reset                           | Forgot <code>drop=True</code> in <code>.reset_index()</code>                           | Use <code>.reset_index(drop=True)</code>                                               |
| 7 | Sort/filter result not saved                                  | Did not assign result to a variable                                                    | <code>sorted_df = df.sort_values('col')</code>                                         |
| 8 | <code>SettingWithCopyWarning</code>                           | Chained indexing: <code>df['col'][row] = val</code>                                    | Use <code>df.loc[row, 'col'] = val</code>                                              |

| #  | Error                                             | Cause                                                    | Fix                                                                     |
|----|---------------------------------------------------|----------------------------------------------------------|-------------------------------------------------------------------------|
| 9  | <code>KeyError: 'product'</code>                  | Misspelled column name                                   | Check <code>df.columns.tolist()</code> , fix the typo                   |
| 10 | <code>KeyError: ('col1', 'col2')</code>           | Passed string tuple instead of list for multiple columns | Use <code>df[['col1', 'col2']]</code> (list inside brackets)            |
| 11 | <code>FileNotFoundError</code>                    | Wrong file path for <code>pd.read_csv()</code>           | Check <code>os.getcwd()</code> , use full absolute path                 |
| 12 | <code>UnicodeDecodeError</code>                   | File not UTF-8 encoded                                   | Try <code>encoding='latin1'</code> or <code>encoding='cp1252'</code>    |
| 13 | <code>IndexError: out-of-bounds with .iloc</code> | Position exceeds DataFrame size                          | Check <code>df.shape</code> , use <code>df.iloc[-1]</code> for last row |

## KEY TAKEAWAYS

**WHY PANDAS:** Labeled, heterogeneous, real-world data tool. Excel has limits; NumPy lacks labels; Pandas solves both.

**SERIES:** 1D labeled array (index + values). The building block of DataFrames. All NumPy operations work, but results keep labels.

**DATAFRAME CREATION:** From dicts (most common), lists of lists, NumPy arrays, or lists of dicts. Dictionary keys become column names.

**READING DATA:** `pd.read_csv()` loads real-world files. Use `usecols`, `nrows`, and `dtype` parameters for control.

**EXPLORATION RITUAL:** `.head()`, `.tail()`, `.shape`, `.info()`, `.describe()` -- ALWAYS run these 5 methods first, every time, no exceptions.

**COLUMN SELECTION:** `df['col']` gives a Series; `df[['col1', 'col2']]` gives a DataFrame. Bracket notation always works; dot notation has pitfalls.

**ROW SELECTION:** `.loc[]` (label, INCLUSIVE) vs `.iloc[]` (position, EXCLUSIVE) -- THE most important distinction in Pandas.

**BOOLEAN FILTERING:** `df[df['col'] condition]` with `&`, `|`, `~` and `.query()`. ALWAYS use parentheses around each condition.

**SORTING:** `.sort_values()`, `.sort_index()`, `.reset_index(drop=True)`. The clean pipeline: filter, sort, reset.

# NEXT SESSION PREVIEW

## Session 4.2: Data Wrangling -- Cleaning, Merging & GroupBy

| Topic                 | What You Will Learn                                                                 |
|-----------------------|-------------------------------------------------------------------------------------|
| Handling Missing Data | <code>.isna()</code> , <code>.fillna()</code> , <code>.dropna()</code>              |
| Data Type Conversion  | <code>.astype()</code> , <code>pd.to_datetime()</code>                              |
| Merging DataFrames    | <code>pd.merge()</code> , <code>pd.concat()</code> (like SQL JOINS)                 |
| GroupBy Operations    | <code>.groupby().agg()</code> -- "Total sales by city", "Average price by category" |

### Why it matters:

- Today you learned to SELECT and FILTER data
- Next session you learn to TRANSFORM and COMBINE data
- Together, these cover 90% of real-world data analysis tasks

**Business Context:** Healthcare Data Integration Failures -- what happens when hospitals merge patient records with mismatched formats.

**Footer:** Vishlesan i-Hub IIT Patna x Masai School